Implementation of a ROS 2 Navigation and Perception Pipeline for TurtleBot 3

Group 29
Ali Shavakhi, Vaheid Labbaf
Email: redacted-for-privacy

***Abstract*—This report presents a modular ROS 2 pipeline for autonomous navigation and perception on the TurtleBot 3 mobile robot in a Gazebo simulation environment. The system includes: (i) dynamic AprilTag-based goal acquisition, (ii) Nav2-based autonomous navigation, (iii) a custom corridor-following strategy, and (iv) LiDAR-based cylindrical table detection with TF transformations. The pipeline leverages Action Clients, Topics, TF, and composable nodes to ensure modularity, efficiency, and robustness.**

***Index Terms*—ROS2, TurtleBot3, Nav2, AprilTag, LiDAR, TF2, Perception, Autonomous Navigation**

I. Introduction

Autonomous mobile robots rely on the tight integration of perception, localization, and navigation to operate effectively in structured indoor environments. In this project, we implement a modular ROS 2 pipeline for the TurtleBot 3 simulation platform, satisfying assignment requirements while reflecting design patterns commonly used in real robotic systems.

The system enables the robot to: (i) extract a dynamic goal from two AprilTags, (ii) navigate using the Nav2 stack, (iii) switch to a custom corridor-following controller in narrow passages, and (iv) detect cylindrical tables through LiDAR-based clustering. The design emphasizes modularity, reproducibility, and standard ROS 2 communication interfaces.

Although developed entirely in simulation, the pipeline mirrors typical perception–navigation architectures, demonstrating how high-level planning, reactive control, and sensor-based detection interact within a unified ROS 2 framework.

II. System Architecture

The system is composed of four independent ROS 2 nodes, each implementing a single responsibility. This follows the principles of decoupling and maintainability commonly emphasized in robotic architectures such as behavior trees and sense–plan–act pipelines.

A. Node-Level Modularity

Each node is responsible for one autonomous functionality:

- **AprilTagNode**: interprets camera/TF data and estimates a stable midpoint target between two AprilTags.
- **NavigationNode**: manages Nav2 lifecycle, sends goals, retries failures, and produces completion signals.
- **CorridorController**: performs low-level manual control using LiDAR when Nav2 becomes unreliable in narrow passages.
- **TableDetector**: executes point cloud segmentation from LaserScan and extracts cylindrical table positions.

This modular breakdown enables independent debugging and integration testing of each subsystem.

B. Communication Pattern

The system uses:

- **Topics** for continuous data flow (e.g., `/scan`, `/tables`),
- **TF transforms** for frame alignment,
- **Actions** for navigation goals (`NavigateToPose`),
- **Lifecycle services** for Nav2 initialization control.

C. Node Summary

Table I summarizes the main inputs and outputs of each node.

TABLE I
ROS 2 Node Architecture Overview

Node	Function	Input	Output
AprilTagNode	Midpoint goal	TF	`/midpoint`
NavigationNode	Nav2 manager	`/midpoint`	`/navigation_completed`
CorridorController	Manual control	`/scan`	`/cmd_vel`
TableDetector	LiDAR clustering	`/scan`	`/tables`

A structural view of the system helps clarify how perception, navigation, and control nodes interact. Each node communicates strictly through ROS 2 topics, actions, and TF, ensuring clean modularity. AprilTag perception updates the navigation goal, the corridor controller overrides Nav2 when needed, and the TableDetector activates only after goal completion. Figure 1 summarizes these interactions in a unified UML diagram.

III. Methods

A. AprilTag Midpoint Estimation

Two distinct implementations of the AprilTag processing node have been developed and provided in the Git repository, each demonstrating a specific feature set. Consequently, two separate video demonstrations have been uploaded: one highlighting the stability of the **Kalman Filter** implementation, and the other demonstrating the **Corridor Control (Bonus)** integration using the baseline estimator.

...

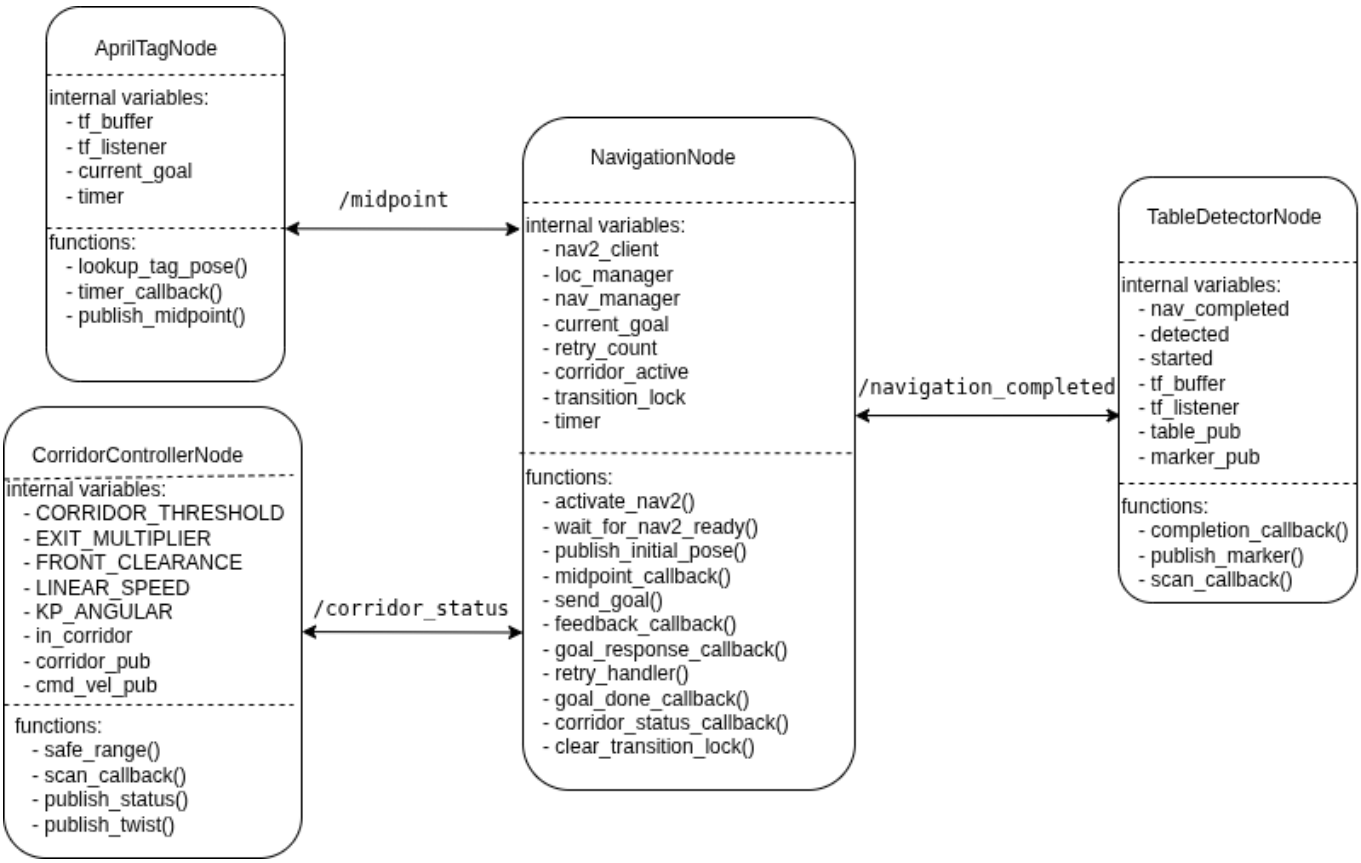


Fig. 1. UML block diagram illustrating internal variables, callback functions, and data flow between AprilTagNode, NavigationNode, CorridorControllerNode, and TableDetectorNode.

Implementation A: Kalman-Filter Based Estimation: The first version augments the perception pipeline with a Linear Kalman Filter. This implementation maintains an internal state vector representing the tag-derived position estimate and propagates it forward using a constant-velocity model. Each new TF lookup serves as a measurement update, significantly reducing noise from rapid fluctuations in tag detections and improving temporal consistency. This version is fully functional and is showcased in the dedicated "Kalman Filter" video to demonstrate superior tracking stability.

Implementation B: Baseline Estimator with Corridor Bonus: The second version, which is utilized for the **Corridor Control (Bonus)** demonstration, employs a lightweight TF-based estimator. This node retrieves the 6-DoF poses of the tags directly from the TF tree and computes the midpoint via arithmetic averaging:

$$x_{\text{mid}} = \frac{x_1 + x_2}{2}, \qquad y_{\text{mid}} = \frac{y_1 + y_2}{2}.$$

To ensure maximum reliability during the complex switching logic required for the corridor following task (the extra point assignment), this baseline estimator was selected for the final integrated bonus pipeline. It suppresses updates when the midpoint changes by less than a fixed threshold to prevent Nav2 re-planning loops.

B. Navigation Pipeline and Lifecycle Management

Before sending any goal, the NavigationNode initializes the Nav2 localization and navigation lifecycle managers. This is essential in ROS 2 deployments, where navigation servers must be brought to the active state in a controlled manner to avoid inconsistent costmaps or uninitialized planners.

Once active, the node subscribes to `/midpoint` and sends the received pose as a `NavigateToPose` action goal. Goal transmission is robustified with:

- a retry mechanism (maximum three attempts),
- feedback handling through the action client,
- success/failure monitoring via the action result.

This allows Nav2 to recover from temporary pose ambiguities or planner timeouts. Upon success, the node publishes a Boolean message on `/navigation_completed` to trigger subsequent perception.

C. Corridor-Control Algorithm

Corridors are a known weak point for purely global planners, where costmaps can shrink and oscillations occur. In the proposed pipeline, the `CorridorController` node

continuously monitors the left and right LaserScan sectors. Corridor entry is detected when both sides fall below $0.55\,\mathrm{m}$, while exit is detected once either side exceeds $0.825\,\mathrm{m}$.

When a corridor is detected, the node:

1) publishes a status flag to pause Nav2,
2) cancels the active navigation goal,
3) enters a manual control mode based on a proportional controller.

The angular velocity is computed as

$$\omega_z = (R - L)\, K_p,$$

where L and R denote the mean left and right LiDAR ranges, and $K_p = -0.05$. A constant forward velocity of $v = 0.35\,\mathrm{m/s}$ is applied as long as the frontal region remains clear. Such a wall-following approach is classic in reactive navigation and works reliably in narrow passages where global planners may fail.

Upon exit, the node publishes that the corridor has ended, the NavigationNode resumes Nav2, and the last goal is resent after a short delay.

D. LiDAR-Based Clustering for Table Detection

After the robot reaches the final target and `/navigation_completed` becomes true, the TableDetector node activates:

1) The incoming `LaserScan` is projected into Cartesian points in the `base_scan` frame.
2) A region-growing algorithm groups consecutive points whose pairwise distance is below $0.4\,\mathrm{m}$.
3) Clusters are filtered by their spatial width; only clusters with width between 0.05 and $0.45\,\mathrm{m}$ are retained, approximating the cylindrical table size.
4) For each cluster, a centroid is computed and transformed into the global `map`/`odom` frame using `tf2`.
5) The resulting positions are published on `/tables` and visualized in RViz using markers.

This detector reliably identifies the three required tables independent of robot orientation.

IV. Results

A. AprilTag Detection Performance

The AprilTagNode consistently detected both tags across all trials, with negligible frame-drop events. TF stability remained within a few millimeters of jitter, enabling a clean midpoint estimate. The dynamic midpoint was reproduced consistently in repeated runs, validating the reliability of the perception subsystem and the robustness of the TF configuration.

B. Navigation and Corridor Traversal

Nav2 successfully planned a valid global path from the spawn pose to the AprilTag-based midpoint goal. In all runs, the corridor was recognized correctly; Nav2 was paused at the entrance of the narrow passage and the custom controller took over.

The P-controller demonstrated smooth behavior, preventing oscillation or side-wall collisions even with noisy LaserScan readings. After detecting the corridor exit, Nav2 was resumed and the previous goal was reissued. The robot completed its navigation to the target without manual intervention or recovery behaviors.

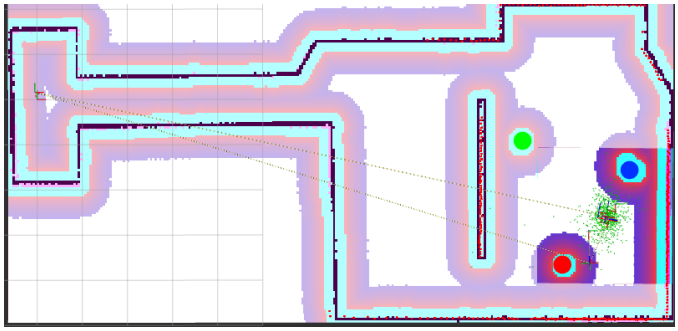


Fig. 2. RViz visualization showing the TurtleBot 3 at the final goal. Three detected cylindrical tables are highlighted using colored markers, alongside the executed Nav2 trajectory and local costmaps.

C. Final Table Detection

Following arrival at the target, the TableDetector node activated and processed LiDAR scans to extract cylindrical clusters. The combination of distance-based clustering and width filtering allowed the algorithm to reliably isolate three clusters corresponding to the tables in the environment.

Figure 2 shows the resulting RViz output, including:

- the robot's final pose at the computed midpoint,
- three segmented tables highlighted with distinct colored markers,
- the global path computed by Nav2,
- local costmaps and LiDAR point cloud around the tables.

V. Conclusion

A complete ROS 2 pipeline was implemented for AprilTag-based goal extraction, Nav2 navigation, corridor traversal, and LiDAR-based table detection. The system operated reliably in simulation and demonstrated clean coordination between perception, navigation, and low-level control. The modular node design also simplified debugging and integration.

Future extensions include deploying the pipeline on a physical TurtleBot 3, improving obstacle handling during corridor traversal, and experimenting with more advanced perception methods for object detection.

Acknowledgements

Git repository
Video demonstration

AI usage disclosure: Portions of the work were supported by the use of AI-based language models (ChatGPT and Google Gemini) strictly for non-creative tasks such as: (i) debugging assistance during the implementation of the NavigationNode startup sequence, and (ii) improving the clarity and formatting of text in the report. All system architecture decisions, algorithms, control logic, perception modules, and ROS 2 code were fully designed and implemented solely by the authors